THEO AI Platform: Developer Brief

Technical handover for the development team. Covers the product, the stack, the architecture and the work to be done, in order.

1. What we are building

THEO is a web-based SaaS platform where a business signs up, picks its industry, and creates its own **AI employee**: an AI receptionist that answers customer questions using that business's own information. The first target industries are **hotels** and **restaurants**. The long-term vision is more industries and more channels (phone, email, messaging), but the first release is deliberately small.

Goal of the first release (Basic plan MVP): a simple, reliable, secure product that real businesses can sign up for and use, so it can be tested with a few real businesses before more is built.

2. Basic plan: what is in scope

- Business registration, login, logout and password reset
- Business onboarding (business name, industry, hours, services, contact details), saved permanently
- Creating and customising **one AI employee** per business: name, personality, job responsibilities, business information
- Business knowledge and FAQ training: the business enters its own information and the AI answers from it
- **Website AI chat**: an embeddable chat widget the business can put on its own website
- Answering customer enquiries and basic booking assistance (collect the booking details and pass them to the business)
- Capturing customer enquiries and messages
- Basic conversation history
- Basic dashboard with simple analytics (conversation count, enquiries captured)
- Subscription billing for the Basic plan

3. Explicitly out of scope for now

These come in later phases. **Do not build them now**, but design the code so they can be added without a rewrite.

- AI telephone calls (a paid add-on later)
- Email handling and inbox integration
- Calendar and booking-system integrations
- WhatsApp and other messaging integrations
- Workflow automation (n8n or similar)
- Multiple AI employees per business (Professional and Premium plans)
- Advanced analytics

4. Starting point and ground rules

- The product already exists as a partly built **FlutterFlow** project: onboarding, AI employee builder, skills library and dashboard screens. It has logic problems (messy signup logic, broken buttons, incomplete Firebase wiring).
- We are **rebuilding on a code-based MERN stack** instead of continuing in FlutterFlow. Recreate the existing screens and flows. Keep the current look, navigation and working features; improve the structure underneath.
- The existing FlutterFlow projects stay **untouched as backups** until the new version is fully working. Never edit the project called "THEO AI Platform backup".
- All code lives in the **private GitHub repository owned by the product owner**. Everything (cloud accounts, database, API keys, domain) is created under the owner's accounts, and the team is invited as members. We never hold the owner's passwords.
- Keep AI usage costs low: use a small model, set limits, and avoid unnecessary test calls.

5. The stack

Layer	Choice	Why
Frontend	React with TypeScript, built with Vite, styled with Tailwind CSS	Fast, simple, easy to match the existing design
Backend	Node.js with Express, TypeScript	Standard MERN, flexible for AI and webhooks
Database	MongoDB Atlas, accessed with Mongoose	Fits per-business documents; free tier to start
Authentication	Own email and password auth: JWT access and refresh tokens in httpOnly cookies, passwords hashed with argon2 or bcrypt	No extra service cost, full control

Layer	Choice	Why
Validation	Zod	Validate every request on the server
AI	OpenAI API with a small, low-cost chat model (for example GPT-4.1 mini or similar)	Cheap, good enough for FAQ-style receptionist chat
Payments	Stripe Checkout, Stripe Customer Portal and Stripe webhooks	Handles subscriptions, upgrades and cancellations
Email	Resend	Verification, password reset and notifications
Hosting	Vercel for the frontend, Render for the backend	Simple deploys from GitHub
Monitoring	Sentry for errors, uptime check on the API health endpoint	Free tiers are enough
Security helpers	helmet, CORS allow-list, express-rate-limit	Basic protection from day one
Source control	GitHub, with `main` for production and `dev` for work in progress	Clear release flow

Not needed for the Basic plan: n8n, Twilio, a voice AI layer, file storage.

6. Architecture

```
Browser (React app)         Customer's website
   |                              |
   | REST (cookies)               | embedded chat widget (script tag)
   v                              v
+--------------------- Express API (Node) --------------------+
| auth | businesses | employees | knowledge | chat | billing |
+----------+-----------------------+-----------+-------------+
           |                       |           |
       MongoDB Atlas            OpenAI       Stripe (webhooks)
```

Two kinds of users interact with the system:

1. **Business owners** use the dashboard (authenticated).
2. **End customers** of those businesses use the chat widget (public, no login, but rate limited and tied to one business by a public key).

7. Multi-tenant design (important)

Every business is a separate tenant. **One business must never be able to read or change another business's data.**

- Every business-owned document stores a `businessId`.
- Every database query for business data **must filter by the `businessId` of the logged-in user**. Take it from the verified token, never from the request body or URL.
- Put this in shared middleware and query helpers so no route can forget it.
- The public chat widget identifies the business through a random, non-guessable `publicKey`, and the server resolves it to a `businessId`.
- Add tests that prove a user from business A cannot read, edit or delete business B's records.

8. Data model (starting point)

User: email, passwordHash, businessId, role (owner), emailVerified, createdAt

Business: name, industry (hotel or restaurant), contact details, opening hours, timezone, publicKey, plan, stripeCustomerId, subscriptionStatus, createdAt

AiEmployee: businessId, name, personality, responsibilities, voice (stored for the future, unused now), skills (array, stored for the future), enabled

KnowledgeItem: businessId, type (faq, service, policy, menu, room), title, content

Conversation: businessId, employeeId, channel (web), visitorId, startedAt, lastMessageAt

Message: conversationId, businessId, role (customer or assistant), content, createdAt

Enquiry: businessId, conversationId, customerName, contact, type (general or booking request), details, status (new, handled), createdAt

Subscription events: store the Stripe event id so webhooks are processed only once.

Add indexes on `businessId` for every collection, and a unique index on `User.email` and `Business.publicKey`.

9. Key flows

9.1 Sign up and onboarding

1. Owner registers with email and password, verifies email.
2. Owner enters business details and picks hotel or restaurant.

3. The system creates the Business, the User and a default AI employee with industry-specific starter instructions.
4. Everything is saved to the database immediately. Closing the browser must never lose progress.

9.2 AI employee setup

Owner edits name, personality, responsibilities and business knowledge (FAQs, services, hours, prices, policies). All changes save to the database. The skills library screen can stay visible but skills beyond Basic are shown as upgrades.

9.3 Chat widget

1. Business copies a one-line script tag from the dashboard.
2. A visitor opens the chat on the business's site and sends a message.
3. The API looks up the business by `publicKey`, loads the AI employee and knowledge.
4. The API builds a system prompt from the employee's personality, responsibilities and the business knowledge, adds the recent message history, and calls OpenAI.
5. The reply is stored and returned to the widget.
6. If the customer wants to book or leave a message, the AI collects their name, contact and details, and the API saves an Enquiry that appears in the dashboard.

Prompt rules: answer only from the business's information; if unsure, say so and offer to take a message; never invent prices, availability or confirmations; never promise a booking is confirmed (in the Basic plan the business confirms it). Keep the knowledge small enough to fit in the prompt for the MVP. If it grows, move to embeddings with MongoDB Atlas Vector Search later.

9.4 Subscriptions

1. Owner chooses the Basic plan in the dashboard.
2. API creates a Stripe Checkout session. Stripe handles the payment page.
3. A Stripe webhook updates `subscriptionStatus` on the Business. **The webhook is the source of truth**, not the browser redirect.
4. The Customer Portal lets owners update their card, cancel or change plan.
5. If the subscription is not active, the widget stops answering and the dashboard shows a clear message.
6. Verify the webhook signature and handle each event only once.

10. API outline

```
POST   /auth/register          POST /auth/login
POST   /auth/logout            POST /auth/refresh
POST   /auth/forgot-password   POST /auth/reset-password
GET    /auth/verify-email

GET/PUT /business              (own business only)
GET/PUT /employee              (own AI employee)
GET/POST/PUT/DELETE /knowledge
GET    /conversations          GET /conversations/:id
GET/PUT /enquiries
GET    /analytics/summary

POST   /billing/checkout       POST /billing/portal
POST   /webhooks/stripe        (raw body, signature verified)

POST   /public/chat            (widget; uses publicKey; rate limited)
GET    /health
```

11. Security checklist

- Passwords hashed, never logged or returned
- JWTs in httpOnly, secure, sameSite cookies; short-lived access token with refresh
- Server-side validation of every input with Zod
- Tenant filter on every business query (see section 7)
- Rate limiting on login, password reset and the public chat endpoint
- CORS restricted to our own domains; widget endpoint allows any origin but only with a valid `publicKey`
- Protect against prompt injection: customer text is never treated as instructions; the system prompt is fixed on the server
- Secrets only in environment variables, never in the repository
- Per-business daily message limit and a monthly spend limit on the OpenAI account
- MongoDB Atlas: restrict network access, use a least-privilege database user, turn on backups once on a paid tier
- Do not store payment card data. Stripe holds it
- Add a privacy policy and terms pages (text provided by the owner)

12. Environment variables

```
MONGODB_URI
JWT_ACCESS_SECRET, JWT_REFRESH_SECRET
OPENAI_API_KEY
STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_ID_BASIC
RESEND_API_KEY, EMAIL_FROM
FRONTEND_URL, API_URL
SENTRY_DSN
```

Keep a `.env.example` in the repository with the names and no values.

13. Accounts the owner creates (we get invited)

GitHub (done), MongoDB Atlas, OpenAI (with a monthly spend limit), Stripe, Vercel, Render, Resend, Sentry and the domain registrar.

14. Suggested repository structure

```
theo-ai/
  client/          React + Vite + Tailwind
    src/pages, components, hooks, api, widget
  server/          Express + TypeScript
    src/routes, controllers, models, middleware,
        services (ai, billing, email), utils
  README.md        setup, scripts, deploy steps
  .env.example
```

Use ESLint and Prettier, small commits, and pull requests from feature branches into `dev`.

15. Work plan

The work is delivered in three phases. Each phase ends with something the product owner can test.

Phase 1: Foundation and authentication

- Set up the repository, project structure, environments and deployments (Vercel and Render)
- Recreate the landing page and existing screens in React, keeping the current look
- Registration, login, logout, email verification and password reset
- Protected routes and session handling

- **Done when:** a new user can register, verify, log in, log out and reach the dashboard shell on the live URL

Phase 2: Business data and AI employee setup

- Business onboarding saved permanently
- Multi-tenant database with the tenant rules in section 7, plus isolation tests
- AI employee builder and knowledge/FAQ editor connected to real saved data
- Dashboard connected to real data
- **Done when:** an owner can onboard a business, configure an AI employee, add knowledge, log out and log back in with everything intact, and a second test business cannot see the first one's data

Phase 3: Working chat, enquiries and billing

- Embeddable website chat widget and the chat API with the prompt rules in section 9.3
- Conversation history and enquiry capture shown in the dashboard
- Basic analytics
- Stripe subscription, webhooks and plan gating
- Final security pass, error monitoring, production deployment and a short handover guide
- **Done when:** a test hotel and a test restaurant each have a live widget answering from their own information, enquiries appear in the dashboard, and a paid subscription turns the service on and a cancellation turns it off

16. Acceptance tests (run before handing over each phase)

1. Register, verify, log in, log out, reset password
2. Refresh the page and reopen the app: session and saved data persist
3. Business A cannot access Business B's data through any route
4. The chat answers correctly from the business's own knowledge and says it does not know when the answer is not there
5. The chat refuses to follow instructions hidden in customer messages
6. A booking request creates an Enquiry with the right details
7. A failed or cancelled subscription disables the widget
8. Stripe webhooks are idempotent (a repeated event changes nothing)
9. The app works on mobile browsers and current desktop browsers
10. No secrets in the repository; production and development use separate keys

17. Design for later phases

Keep these in mind so the Basic plan can grow without a rewrite:

- **Plan and feature flags**: store each business's plan and check features through one helper, so Professional and Premium features can be switched on later.
- **Channels**: store `channel` on every conversation (web now; phone, email and WhatsApp later).
- **Skills as modules**: keep each AI skill as a separate module with its own settings.
- **Multiple employees**: the data model already allows several AI employees per business.
- **Integrations**: put booking, calendar and email integrations behind service interfaces.
- **Phone later**: likely Twilio for numbers and calls plus a voice AI layer, added as a paid add-on.

18. Communication and delivery notes

- Work from the `dev` branch and open pull requests; deploy to production only after the phase checks pass.
- Keep the owner updated with short progress notes and a demo link at the end of each phase.
- Document anything unusual in the README so the owner can maintain the project independently.
- Ask before changing the scope or the design.